

WEEK 7 · DAY 1

Django Views & Multi-Method View System

Compact interactive revision guide — everything from today's lesson + lab.

CORE FLOW

Browser → URL → View → Logic/Data → Response → Browser

Today's topics: HttpRequest · HttpResponse · GET/POST · FBV · CBV · dispatch()
· render() · context · sessions · JsonResponse · URL names

✓ Day 1 lab completed

01 · BIG PICTURE

View = coordinator. It receives the request, does what is needed, and returns a response.

IN THE REQUEST	WHAT IT MEANS
<code>request.method</code>	GET, POST, etc.
<code>request.GET / POST</code>	Incoming query/form data
<code>request.session</code>	Session state
<code>request.FILES / COOKIES</code>	Files / browser cookies
<code>request.headers / path</code>	Metadata / current path

Responses: HttpResponse = simple output · render() = HTML + context · redirect() = another route · JsonResponse = JSON
· FileResponse = file

02 · GET / POST + FBV / CBV

GET → usually retrieve/display.

POST → usually submit data or perform an action.

CBV pattern

```
class LoginView(View): def get(self, request): return render(request, "accounts/login.html") def post(self, request): return HttpResponseRedirect("Login Form Submitted")
```

FBV is a function. **CBV** is a class with methods. In a CBV, **dispatch()** sends the request to the matching handler such as `get()` or `post()`.

Templates + Context

```
context = {"username": "Aly"} return render(request, "home.html", context) # template <h1>Hello {{ username }}</h1>
```

Context = data passed from Python to the HTML template. Think: Python prepares → context carries → template displays.

Sessions

```
username = request.session.get("username", "Guest") return render(request, "accounts/profile.html", {"username": username})
```

The session is checked for **username**. If it is missing, the fallback is **Guest**.

03 · JSON + URLs + LAB

JsonResponse

```
def status(request): return JsonResponse({"status": "OK"})
```

This returns JSON. We tested `/status/` and the browser showed `{"status": "OK"}`.

URL connections

```
urlpatterns = [ path("register/", RegisterView.as_view(), name="register"), path("login/",  
LoginView.as_view(), name="login"), path("profile/", ProfileView.as_view(), name="profile"), path("status/",  
status, name="status"), ]
```

CBV → use `.as_view()` · FBV → pass the function directly.

Day 1 lab checklist

- Create accounts app
- RegisterView GET/POST
- LoginView GET/POST
- ProfileView reads session
- FBV status endpoint
- JsonResponse for status
- URLs with names
- Test pages + screenshots

LAB VERIFIED ✓

Register ✓ · Login ✓ · Profile ✓ · Status JSON ✓

The browser tests we captured today are part of the lab evidence.

04 · ACTIVE RECALL + CHEAT SHEET

Try these first without looking at the answers.

1. What does a View receive and return? → _____
2. Where do query-string parameters live? → _____
3. Which method commonly handles form submission? → _____
4. What are the CBV methods from today? → _____
5. What is context? → _____
6. What does `session.get(key, fallback)` give you? → _____
7. What returns JSON? → _____
8. How does a CBV connect to `path()`? → _____

Answer key

1 Request → Response · 2 `request.GET` · 3 POST · 4 `get()` and `post()` · 5 Data passed from View to template · 6 Stored value, or fallback if missing · 7 `JsonResponse` · 8 `ViewName.as_view()`

ONE-MINUTE CHEAT SHEET

Need	Pattern
HTML page	<code>render(request, "page.html", context)</code>
CBV GET / POST	<code>get(self, request)</code> · <code>post(self, request)</code>
Session	<code>request.session.get("key", "fallback")</code>
JSON	<code>JsonResponse({"status": "OK"})</code>
CBV URL	<code>LoginView.as_view()</code>
FBV URL	<code>status</code>

DAY 1 COMPLETE ✓

Views · GET/POST · FBV/CBV · Templates · Context · Sessions · JSON · URLs · Browser testing

Study routine: Read → cover answers → explain Browser → URL → View → Response aloud → re-type the core patterns.